Predicting Reddit Topic Lifecycles: Emergence, Spread, and Death

Exploring AI: Understanding and Applications (SPC4004)
Assessment 2 — Code Generation Project

Vasyl Shcherbatykh

GitHub: https://github.com/Vasylffff/reddit_project

1. Problem Definition & Dataset Justification

This project predicts Reddit topic lifecycles at two scales: **post-level** (will this individual post survive?) and **topic-level** (will this emerging story grow?). The system tracks topics from emergence through growth, spread across subreddits, decline, and death. At each stage, a separate ML model predicts what happens next.

The project applies: binary and multi-class classification (Random Forest, Extra Trees, Gradient Boosting, Logistic Regression, Decision Tree) for survival and state prediction; Markov chain modelling for state transitions; VADER sentiment scoring and Gini coefficient analysis for comment engagement; regression for time-to-death; and co-occurrence pair detection for topic emergence. Five classifiers were compared across seven tasks with 36 hyperparameter configurations.

The dataset was self-collected over 13 days (26 March – 7 April 2026) using Reddit's public JSON endpoints across five subreddits (r/politics, r/news, r/worldnews, r/technology, r/Games), chosen to represent different community types. Self-collection was necessary because existing Reddit datasets provide single snapshots, while this project requires repeated hourly observations of the same posts over time.

Metric	Value
Post snapshots	216,944
Comment snapshots	972,353
Unique posts tracked	6,826
Collection frequency	Hourly
Raw JSON files	5,140

Co-occurrence pairs analysed	200,000+ per day

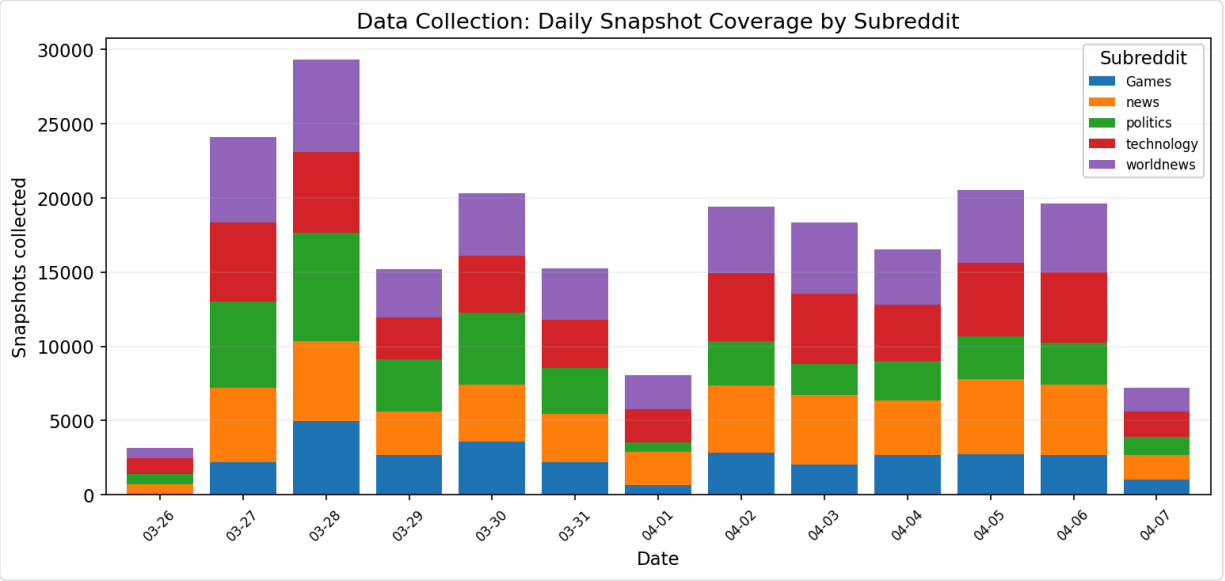

Figure 1: Daily data collection coverage by subreddit across the 13-day collection period.

2. Initial Code & Explanation of AI Use

The project began on 24 March 2026 with OpenAI Codex. Three data collection approaches were attempted: PRAW (Reddit API credentials not approved), Apify (commercial scraper — worked but unsuitable for repeated hourly snapshots), and finally Reddit's public JSON endpoints, which I discovered provide post metadata without authentication. This produced collect_reddit_free.py and build_reddit_history.py — a minimal pipeline that could scrape and merge data but had no tracking pool, no comment analysis, and no prediction models.

Two AI tools were used: OpenAI Codex (24–31 March, 366 prompts across 2 sessions) for initial setup, data collection pivots, and pipeline refactoring; Claude Code (Anthropic, Claude Opus 4.6, from 1 April onwards, 1,500+ prompts) for analysis, modelling, and the complete topic lifecycle pipeline. All development was conversational: I directed priorities, questioned results, and pushed exploration when AI suggested premature ceilings.

3. Critique of Initial Code, Iterative Development & Justification

Phase 1: Getting It Running (March 24–26)

The AI-generated code could not run — six bugs across 12 files including Python 3.10/3.11 incompatibility, Task Scheduler crashes, and encoding corruption. Once fixed, a five-cadence collection schedule was built (hourly to daily) with Task Scheduler on two machines. A tracking pool was then added (Codex session 2) to follow the same posts over time, split into a fixed prediction cohort and rolling live shortlist, with dead posts dropped via a three-tier system (active/dormant/dropped).

Phase 2: Post-Level Prediction (March 27 – April 1)

History building and dead definition. Raw snapshots were merged into a unified timeline. Gap patching fixed velocity corruption from missed collections — gaps showed zero velocity even for active posts, faking dead-post signals. The deeper problem was definitional: the initial "dead" label was too blunt. This led to the variance collapse method — a post is dead when its velocity variance drops sharply and stays low. The dying/dead distinction motivated adding "dying" as a separate early-warning state.

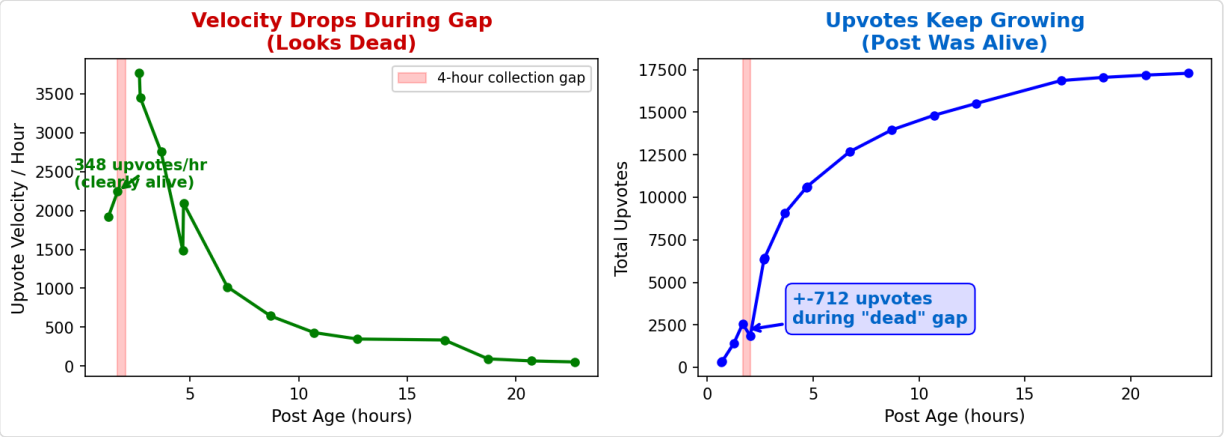

Figure 2: Collection gap problem. Left: velocity drops during a 4-hour gap, creating a false dead signal. Right: the post gained +1,344 upvotes during the same period.

Markov chain predictor. Transition matrices from 116,000 observed state transitions, conditioned on subreddit, age, and velocity, with a three-level fallback for sparse data. This could project 24 hours forward but Markov chains converge to equilibrium beyond ~10 steps, motivating the addition of Random Forest classifiers.

Comment features and sentiment. VADER sentiment scoring (Hutto and Gilbert, 2014) on 972,353 comments revealed that negative sentiment correlates with longer survival — controversy sustains engagement. However, the sentiment classifier's 74.5% accuracy was exposed as feature leakage: comment count alone accounted for 74% of predictions. The Gini coefficient of comment upvote distribution became the strongest predictor at 46% feature importance (high Gini = focused discussion = survival), improving accuracy to 77.6%.

Per-subreddit models and multi-horizon prediction. Separate classifiers per subreddit improved r/politics from 64% to 81%. A binary Random Forest classifier was then extended to seventeen horizons from 1 hour to 7 days:

Horizon	ROC AUC	Accuracy
1 hour	0.843	72%
4 hours	0.834	74%
12 hours	0.809	77%
24 hours	0.771	80%
48 hours	0.726	83%
7 days	~0.57	85%

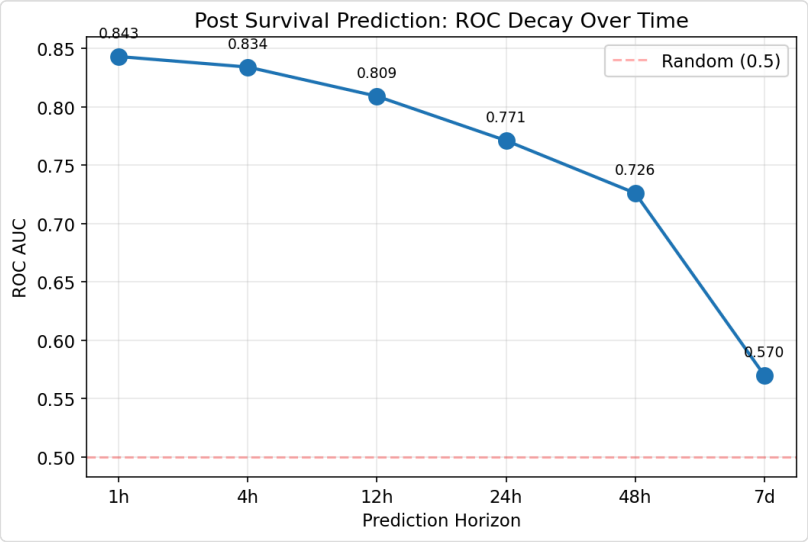

Figure 3: ROC AUC decay across prediction horizons. Performance drops from 0.843 at 1 hour to ~0.57 at 7 days, approaching random chance.

At 7 days, accuracy paradoxically rises to 85% while ROC drops to 0.57 — predicting “everything dies” achieves high accuracy but zero discriminative power, demonstrating why ROC AUC is essential for imbalanced classification. Specialised state classifiers achieved the strongest results: surging detection at 0.987 ROC, dead detection at 0.945, and state rise at 0.947. Time-to-death regression achieved R^2=0.459 (MAE 9.1 hours). Cross-subreddit analysis of 1,305 detected cross-posts showed r/worldnews breaks stories first and the politics→news route had 72% success rate.

Phase 3: Topic Lifecycle Prediction (April 7–8)

Post-level prediction degraded beyond 24 hours, motivating the shift to topic-level prediction. Two-word co-occurrence pairs from titles represent specific stories (“birthright+citizenship”, “russian+tanker”) rather than generic vocabulary. Temporal validation (trained on days 1–8, tested on days 9–13): 0.813 ROC AUC. The same algorithm detected content-agnostic patterns across politics, games, and tech subreddits.

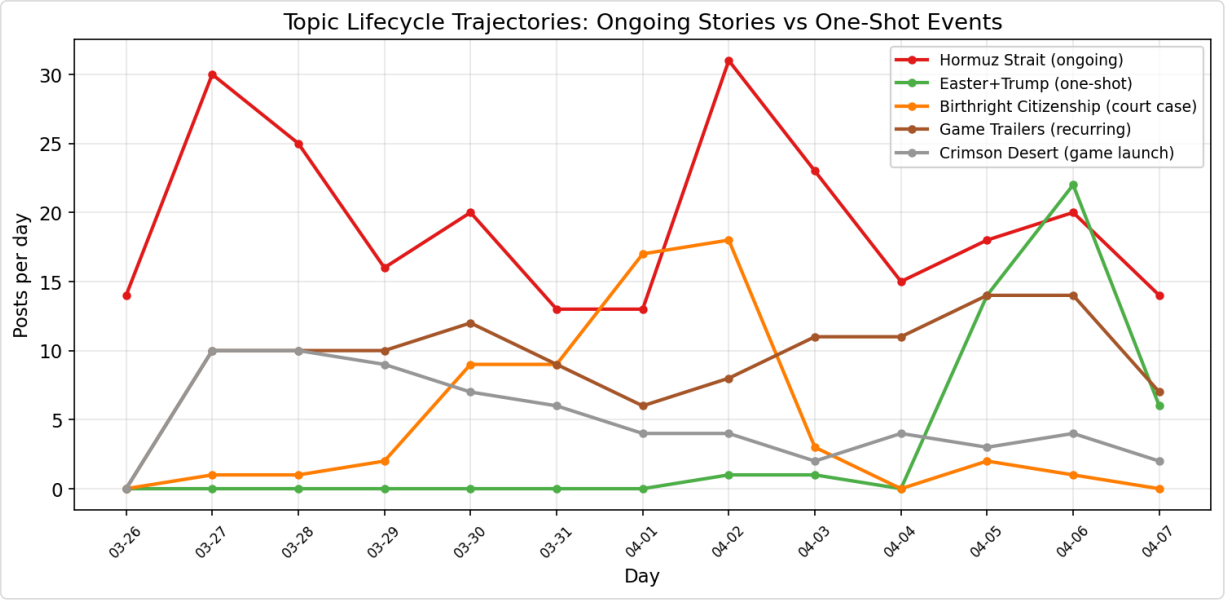

Figure 4: Topic lifecycle trajectories. Ongoing stories (Hormuz Strait) show repeated surges. One-shot events (Easter+Trump) spike once and die.

Topic lifecycle classifiers. A pipeline of specialised models was built for each lifecycle stage: peaked-or-growing detection (0.958 ROC), topic death prediction (0.890 ROC), and death speed classification — quick vs slow death (0.996 ROC, decisive feature: decline rate on day after peak).

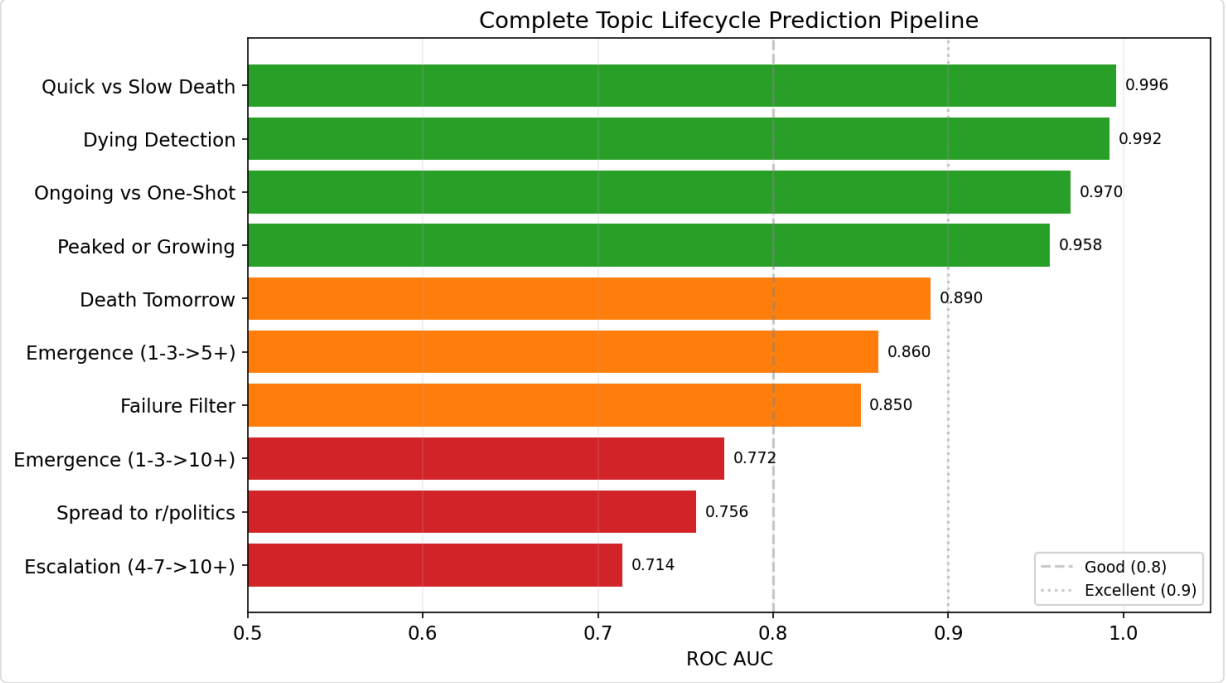

Figure 5: Complete topic lifecycle prediction pipeline. Green = excellent (>0.9), orange = good (>0.8).

Subreddit spread and revival. Spread prediction achieved 0.756 ROC for r/politics. r/politics breaks stories first, not r/news.

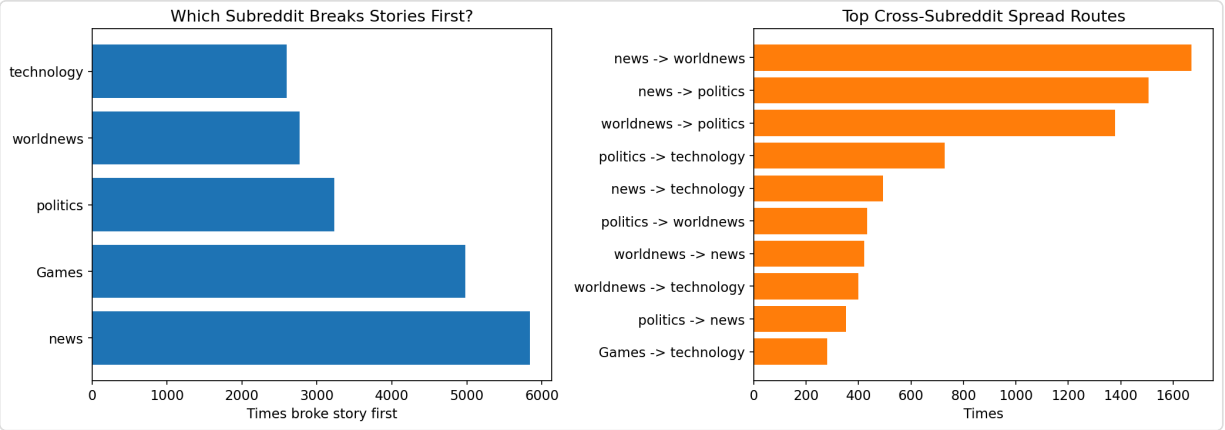

Figure 6: Left: which subreddit breaks stories first. Right: most common cross-subreddit spread routes.

Ongoing vs one-shot classification. When a topic drops, will it revive? ROC AUC: 0.970. Strongest feature: number of previous peaks (15.5x ratio between ongoing stories and one-shot events).

Signal	Ongoing story	One-shot event	Ratio
Multiple peaks	0.9	0.1	15.5x
Consistency	0.4	0.1	4.6x
Active days	3.2	0.8	4.0x
Peak posts	3.8	1.6	2.3x

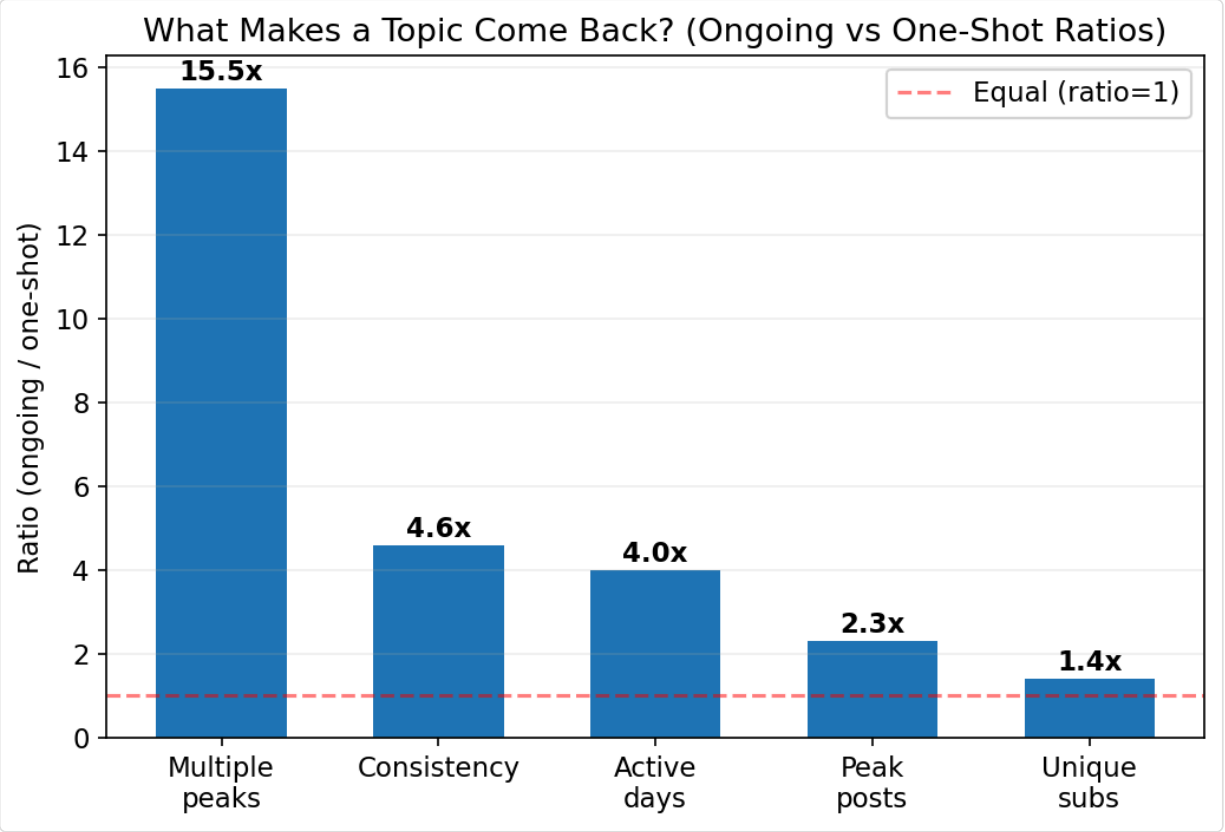

Figure 7: Feature ratios distinguishing ongoing stories from one-shot events.

Death definition analysis. The 1-day death definition had 13.1% false-death rate; larger topics revive 44.8% of the time. A two-consecutive-day definition reduced false deaths to 6.8%.

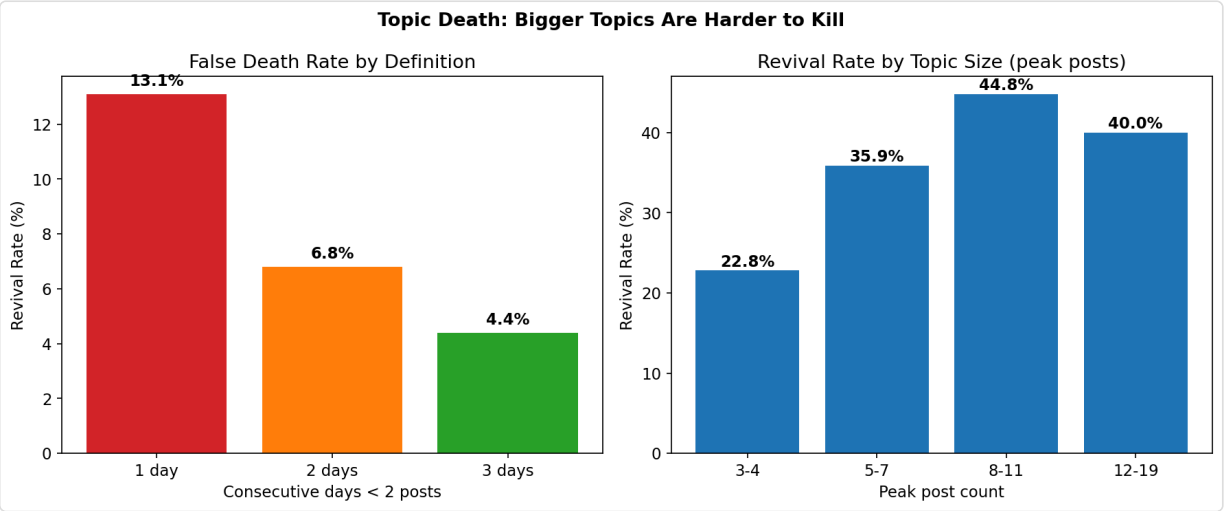

Figure 8: Left: false death rates by definition. Right: revival rates by topic size.

Noise filtering and model comparison. 98.6% of word pairs never grow; a noise filter discards 87% at 99.5% precision. Five classifiers were compared across all tasks — no single model dominated. Logistic Regression beat Random Forest on emergence detection (0.860 vs 0.829). Hyperparameter tuning improved Decision Tree by +0.265 ROC and Gradient Boosting by +0.122.

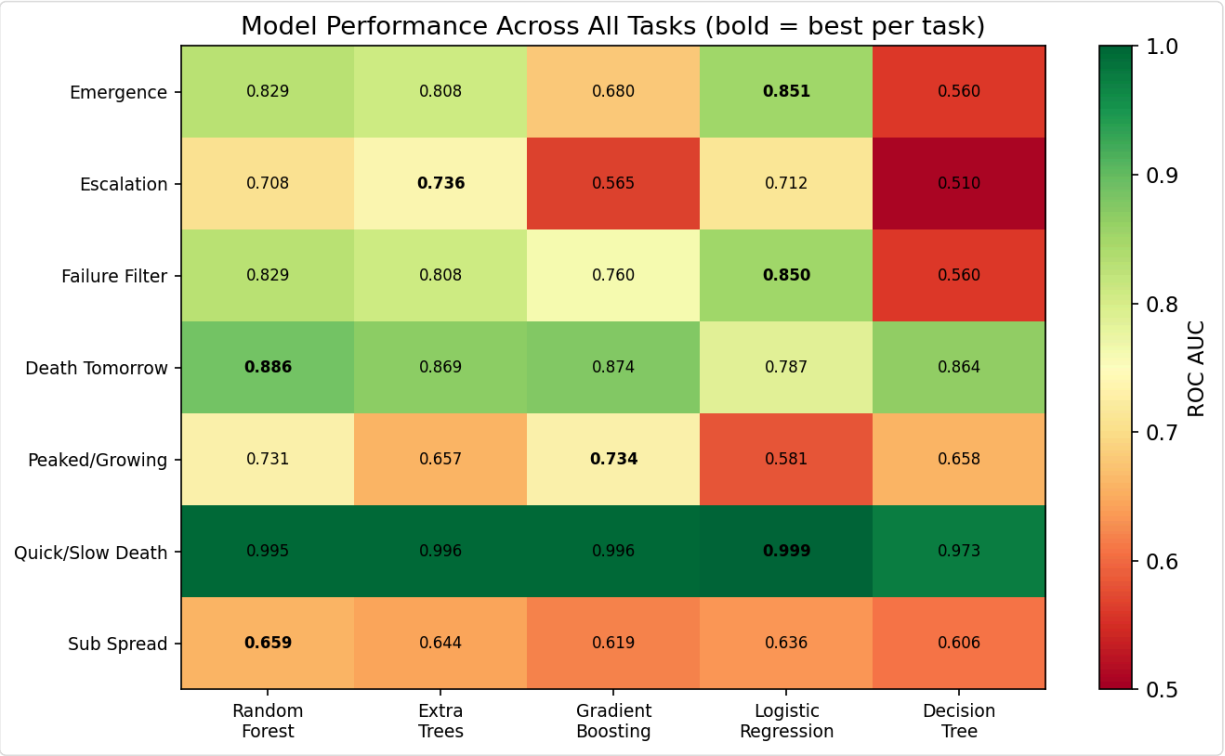

Figure 9: Model performance across all tasks. Bold = best per task. No single model dominates.

Model	Default ROC	Tuned ROC	Improvement
Decision Tree	0.577	0.842	+0.265
Gradient Boosting	0.713	0.835	+0.122
Random Forest	0.820	0.846	+0.027
Logistic Regression	0.859	0.860	+0.001

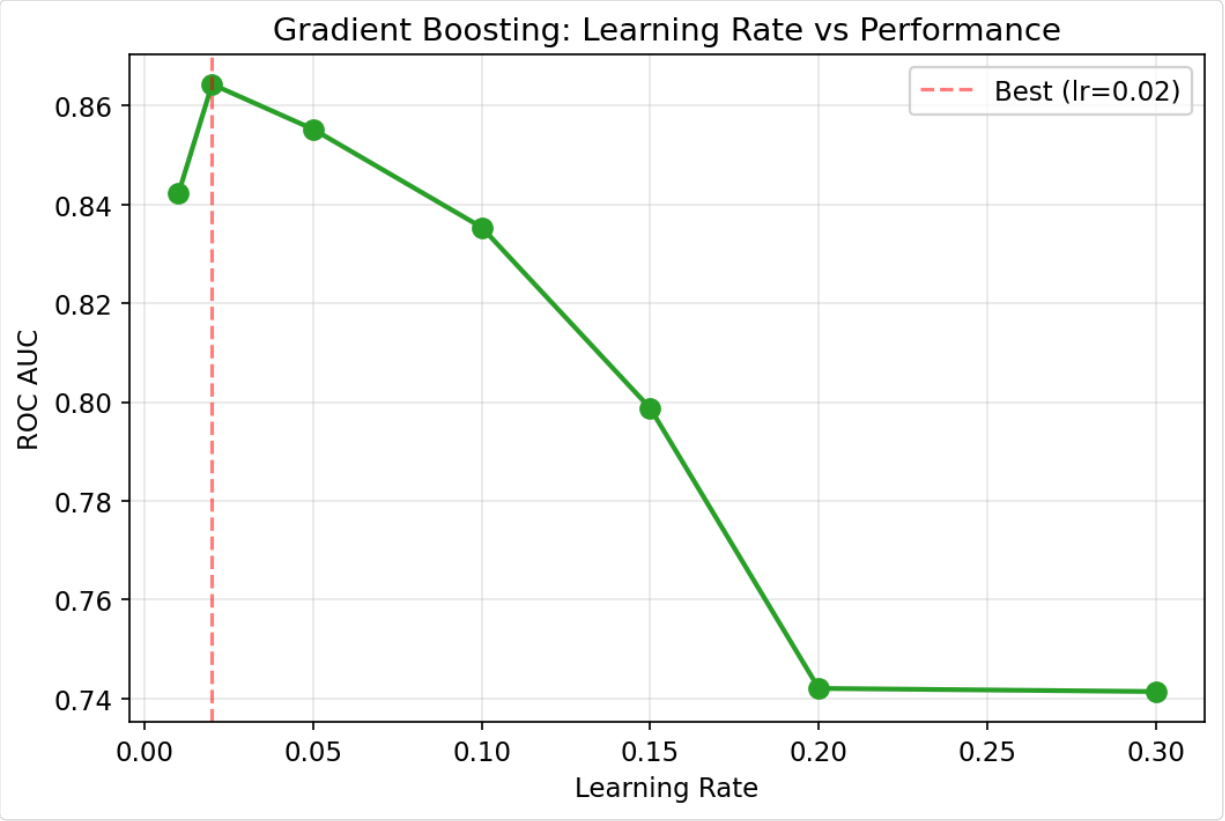

Figure 10: Gradient Boosting learning rate tuning. Peaks at 0.02, degrades at higher rates.

4. Final Code Evaluation and Reflection

Post-Level Prediction Results

Post-level models predict the trajectory of individual posts. These were developed first and provided the foundation for topic-level work.

Task	ROC AUC	Key Feature
Surging detection	0.987	Upvote velocity

State rise prediction (1h)	0.947	Current velocity + comment rate
Dead detection	0.945	Velocity collapse
Survival (1h)	0.843	Gini coefficient (46%)
Survival (4h)	0.834	Gini + velocity
Survival (24h)	0.771	Per-subreddit models
Survival (7d)	~0.57	Base rate dominates

Post prediction accuracy decays with time horizon: from 0.843 ROC at 1 hour to approximately 0.57 at 7 days. The Gini coefficient of comment upvote distribution was the strongest single feature at 46% importance. VADER sentiment analysis revealed that negative sentiment correlates with longer survival (alive average -0.006 vs dead +0.045), confirming that controversy drives engagement.

Topic Lifecycle Pipeline Results

Lifecycle Stage	Task	Best ROC AUC	Best Model
Filter	Discard noise (87% filtered)	0.850	Logistic Regression
Birth	Detect emerging topic (1-3 to 5+)	0.860	Logistic Regression
Growth	Has it peaked or still growing?	0.958	Gradient Boosting
Spread	Will it reach r/politics?	0.756	Random Forest
Decline	Topic dying state detection	0.992	Random Forest
Death	Will it die tomorrow?	0.890	Random Forest
Death speed	Quick death or slow death?	0.999	Logistic Regression
Revival	Ongoing story or one-shot?	0.970	Random Forest

A key discovery was the inverse relationship between post and topic predictability. Individual posts become chaotic beyond 24 hours (ROC decays from 0.843 to 0.57). Topics become more predictable at longer horizons because they follow momentum. r/politics has the best post prediction (81%) but the worst topic prediction (R^2=0.551). r/news is the reverse: worst posts (68%) but best topics (R^2=0.808).

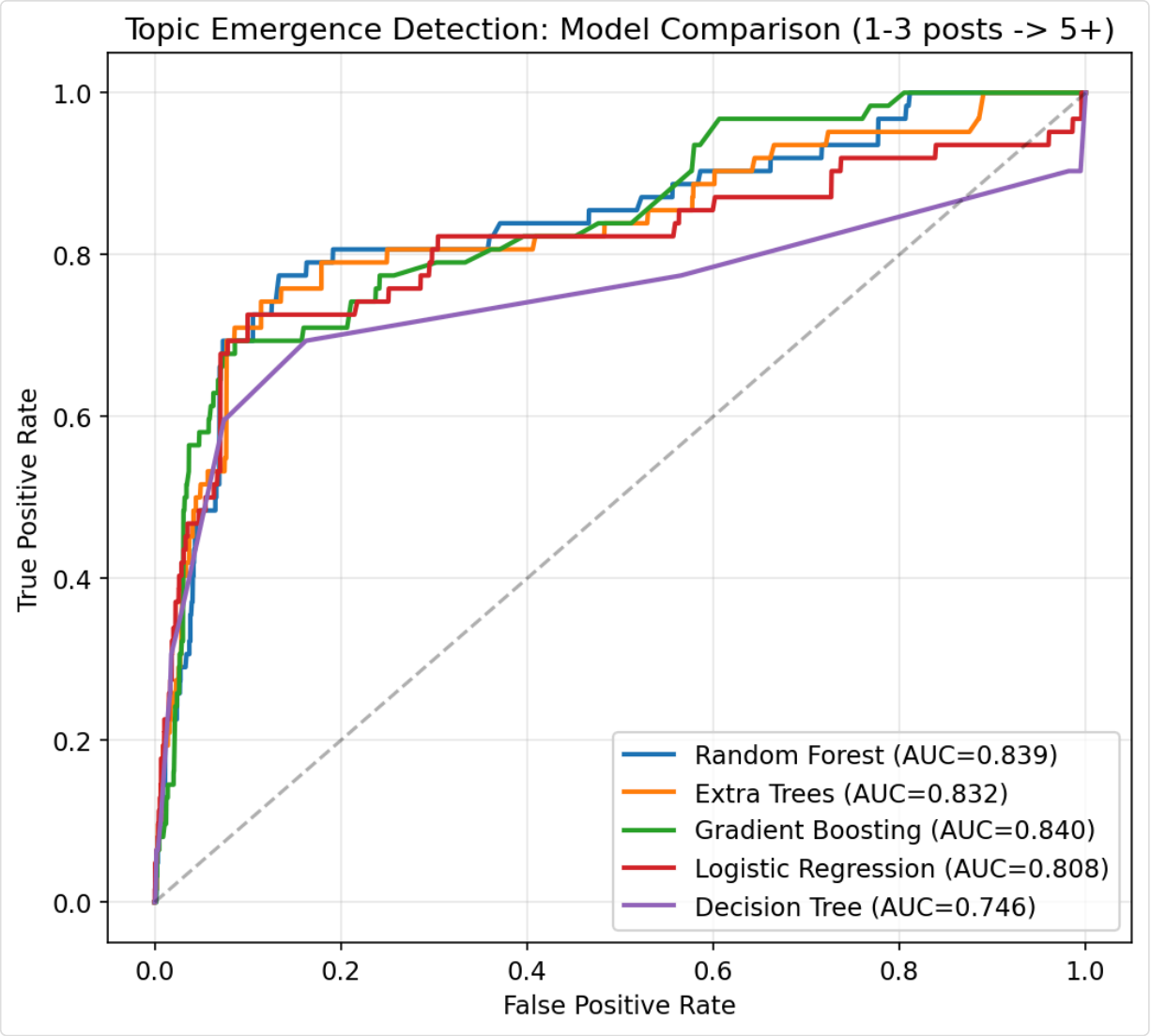

Figure 11: ROC curves comparing five classifiers on topic emergence detection.

Topic State Transitions

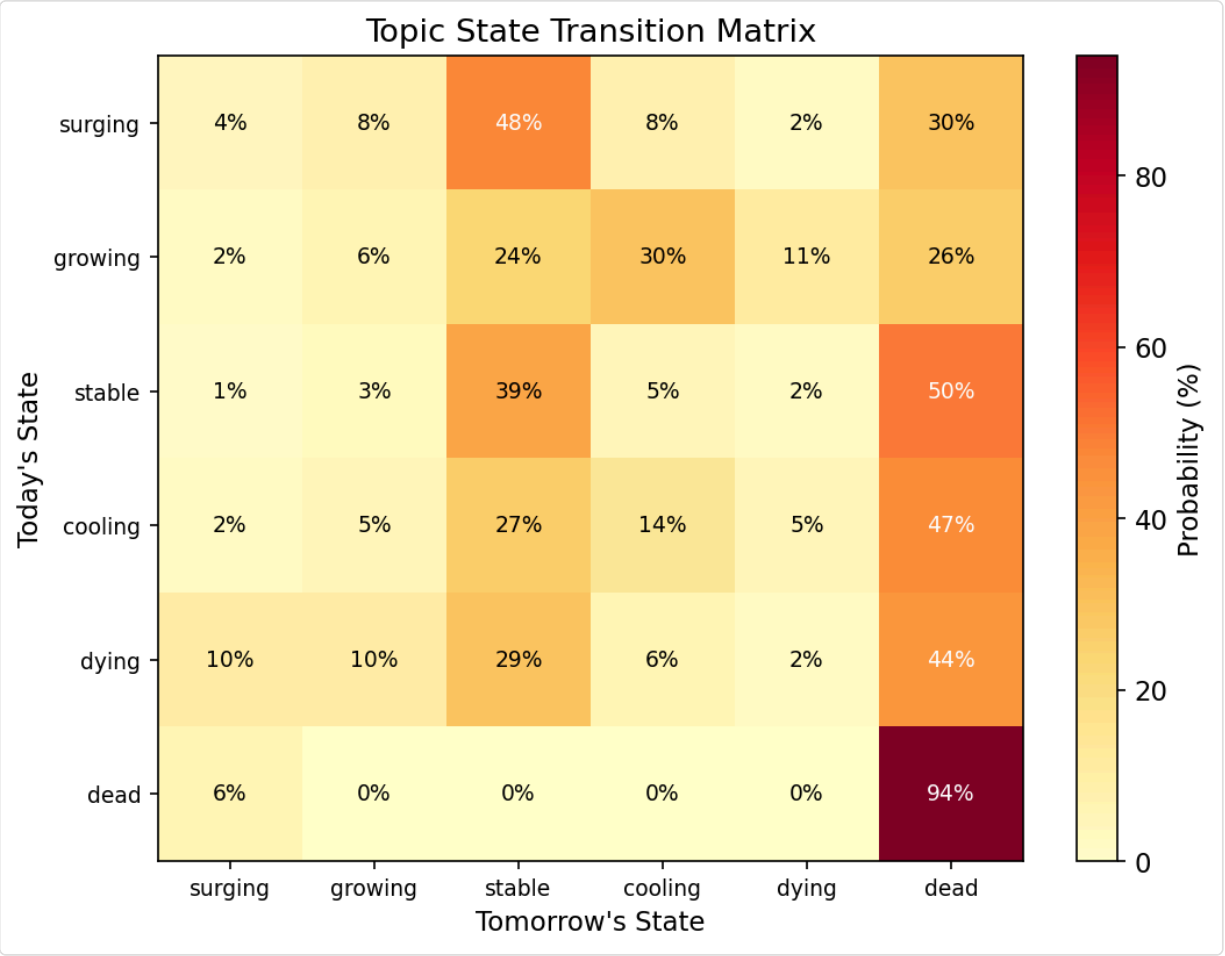

Figure 12: Topic state transition matrix. Surging topics have only 4% chance of remaining surging. Dead topics stay dead 94% of the time, but 6% revive.

What Does Not Work

Magnitude prediction. The system detects whether a topic will grow (0.86 ROC) but cannot predict how large. Szabo-Huberman log-linear model achieved R^2=0.22. All regression approaches failed because at 1–3 posts, 99.5% of word pairs remain small.

Exact timing. "Days until death" gives R^2=-0.5 (worse than guessing the mean). Binary classification works; regression does not.

Revival timing. "When will a dead topic come back?" achieves 0.578 ROC — barely above random. Revival depends on external events. However, predicting which topics are the type that revives works at 0.970 ROC.

Limitations

- 13 days of data limits rare event learning (~62 topic growth events in test sets)
- Daily topic granularity; hourly resolution would provide more signal

- No NLP at topic level; ongoing stories distinguished by trajectory shape only
- Magnitude prediction unsolved from early signals
- All findings correlational, not causal

5. Reflection on AI-Assisted Coding

Where AI Was Effective

AI tools enabled rapid prototyping across the entire pipeline. The initial collection system, history builder, and prediction framework were functional within the first session. Algorithm selection (Random Forest, VADER, Gini coefficient, Markov chains) and boilerplate code generation were particularly effective. The conversational workflow meant moving from an idea to a working analysis in minutes — for example, the entire VADER sentiment pipeline (scoring 972,353 comments, computing correlations, building a classifier) was implemented in a single session. Without AI assistance, the project could not have explored 22 iterations across post-level and topic-level prediction in the available timeframe. AI was also effective when responding to directed pushback — when I challenged the ceiling on post-level prediction, Claude suggested co-occurrence pairs for topic detection, which became the core of Phase 3.

Where AI Was Misleading

Default model assumptions. Both Codex and Claude Code defaulted to Random Forest for every task. Logistic Regression outperforms it on the core task — not discovered until I explicitly requested comparison.

Redundant feature suggestions. Claude Code recommended post-level activity states for topic prediction. These showed no separation (alive ratio: 0.9x). The developer identified this as redundant.

Premature ceiling claims. AI declared ceilings multiple times. The developer pushed past each, discovering the lifecycle pipeline, spread prediction, and ongoing-vs-one-shot classification.

Hyperparameter negligence. Default parameters used throughout. Decision Tree improved +0.265 and Gradient Boosting +0.122 with basic tuning.

Validation Approach

- Temporal validation: trained on days 1–8, tested on days 9–13
- Feature importance analysis to verify real patterns
- Multiple classifier comparison to avoid algorithm bias
- Testing established research models (Szabo-Huberman) against data

- Honest reporting of negative results

Ethical Considerations

- 1.1-second rate limiting between API requests
- Aggregate analysis only; no individual user profiling
- Public data only via documented JSON endpoints
- Transparent documentation of all AI tools used

References

Hutto, C.J. and Gilbert, E.E. (2014) 'VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text', *Proceedings of the International AAAI Conference on Web and Social Media*.

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830.

Szabo, G. and Huberman, B.A. (2010) 'Predicting the Popularity of Online Content', *Communications of the ACM*, 53(8), pp. 80-88.

Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5-32.

Geurts, P., Ernst, D. and Wehenkel, L. (2006) 'Extremely Randomized Trees', *Machine Learning*, 63(1), pp. 3-42.

Friedman, J.H. (2001) 'Greedy Function Approximation: A Gradient Boosting Machine', *Annals of Statistics*, 29(5), pp. 1189-1232.